Traceability ≠ Clinical Correctness

Sep Agent · Chain-of-Thought + RAG · ejection-fraction characterization reliability — a 4-round measurement & fix investigation

---

CURRENT STATUS: FLAG-SPECIFIC ANCHOR ADOPTED (ROUND 4)

Error rate reduced from 12.0% (round-2 baseline) to 4.0% — real improvement, not full elimination

Overview: Four Rounds of Measurement

All rounds use the same N=25 reliability sample (5 ACDC patients × 5 repeats each, CoT+RAG production config) for direct comparability. Round 2's 12.0% is the accurate baseline against which rounds 3 and 4 are measured (round 1's lower 8.0% was partly an artifact of truncated generations masking errors before they could fully materialize — see Round 2 below).

Round	Change tested	Error rate	Outcome
1	Original 2 checks (directional + citation), default settings	8.0% (2/25)	Superseded - truncation masked errors
2	+ fixed num_ctx (Ollama context window truncation bug)	12.0% (3/25)	Accurate baseline for rounds 3-4
3a	temperature = 0.2, no seed	16.0% (4/25)	Rejected - made things worse
3b	temperature = 0.2 + per-patient fixed seed	0.0% (0/25)	Not adopted - reproducibility caveat
4	+ flag-specific prompt anchor (default temperature)	4.0% (1/25)	ADOPTED - current production config

ROUND 4 — ADOPTED

Flag-Specific Prompt Anchor

Round 3 showed temperature alone made things worse, and temperature+seed's 0% result was likely partly a small-sample/seed-non-reproducibility artifact rather than a structural fix (see Round 3 below) — so a prompt-level fix targeting the diagnosed root cause was tried instead.

`_build_directional_anchor_block` (`sep_agent.py`) injects an explicit, per-case reminder for each directional flag actually present, naming the flag and its forbidden words directly — config externalized to `src/directional_flags_config.json` since this note was first written, one entry per flag with its measure terms, forbidden words, AND anchor description together, so the check and the prompt anchor can't drift apart — e.g. for `high_lv_ejection_fraction_suspected`: *"the LEFT ventricular ejection fraction is ELEVATED (hyperdynamic / preserved-high)... never use these words: 'reduced', 'decreased', 'diminished', 'depressed', 'low', 'normal'."* This is stronger than the pre-existing generic direction rule (kept unchanged) because it names the specific flag and specific words, not just a general instruction to "be consistent with flags."

Outcome	Round 2 (baseline)	Round 4 (+ anchor)
Clean on first attempt	19/25	15/25
Clean after a retry	3/25	9/25

Outcome	Round 2 (baseline)	Round 4 (+ anchor)
Exhausted retry budget	3/25	1/25
Final-output error rate	**12.0%**	**4.0%**

Real improvement, not elimination. All 3 round-2 exhausted failures were `high_lv_ejection_fraction_suspected` — with the anchor, that flag had zero exhausted failures (patient102: 3/5 repeats still needed a retry for it, but all 3 now recovered instead of 2/5 exhausting). The one remaining round-4 failure is a **different** flag — patient109, `low_rv_ejection_fraction_suspected`, calling 33.7% RV EF "normal" across all 3 attempts — despite the same anchor mechanism applying to this flag too. So the anchor substantially reduces, but does not guarantee against, this failure class, even for the exact flag it was written for.

Not yet tried: combining the anchor with the round-3 temperature finding, or extending this config-file approach to flags outside the current 5 (e.g. `pathology__pattern_suspected`, which has no directional check at all yet) — adding one is now a JSON edit (`src/directional_flags_config.json`), not a code change.

ROUND 3 — TESTED, NOT ADOPTED

Does Lowering Temperature (and Fixing a Seed) Help?

Tested empirically, not adopted on intuition — same discipline as the embedding-model comparison. Two things confirmed first, directly: (1) medgemma:4b's own Modelfile sets no `PARAMETER temperature` or `seed` — only `stop` — so every prior round used Ollama's own runtime default (~0.8); (2) 3 identical short calls with no options produced 3 different phrasings (non-determinism confirmed), while 2 identical calls with an explicit seed returned byte-identical text on that same short prompt.

Config	Clean 1st try	Clean after retry	Exhausted	Error rate
Default (~0.8, no seed) - round 2	19/25	3/25	3/25	12.0%
Temperature=0.2, no seed	19/25	2/25	4/25	16.0% - WORSE
Temperature=0.2 + per-patient fixed seed	21/25	4/25	0/25	0.0%

Temperature alone made it worse, not better — confirmed, not just theorized. 3 of the 4 exhausted failures at temperature=0.2 were the exact same pattern as round 2 (elevated-EF characterization errors on patient104/103/102). This is the predicted failure mode of sharpening the distribution when the model's underlying bias already favors the WRONG word for this flag ("reduced"/"normal" more probable than "elevated" in its training distribution for this phrasing) — lower temperature makes the model pick its most likely token more reliably, which is actively harmful when the most likely token is wrong.

Temperature + seed reached 0/25 exhausted failures — but with an important caveat found while checking the result, not assumed. The seed did NOT produce byte-identical output at the length/complexity of real production prompts, despite doing so on the short test prompt: only 1 of 5 patients (patient103) had fully identical text across all 5 repeats; the other 4 had 3-5 distinct summaries each, sometimes substantively different. The directional/citation check DID fire 4 times even in this configuration — it just always succeeded within the retry budget this time, unlike round 2. 0% is a genuine, positive result on this sample, **not a guarantee the model became deterministic** — likely floating-point non-associativity in batched GPU inference compounding over a long generation (a known llama.cpp/Ollama characteristic, not unique to this project).

Not adopted as the production default. Two reasons for caution: (1) N=25 with only 5 distinct seeds is a small sample to generalize a 0% claim from; (2) since temperature alone made things measurably worse, any future change to the prompt or checks could shift the model's probability landscape again, untested against this specific temperature+seed combination.

ROUND 2 — BUG FIX

Fixing MedGemma's JSON Truncation (num_ctx)

While validating round 1 (below), 9 of 13 retries and both exhausted failures turned out to share one root cause unrelated to this limitation: `agent_common.generate_report_medgemma` sent no `options` to Ollama, so it fell back to its small default context window (~2-4k tokens) instead of using medgemma:4b's real 131k-token capacity — with a long CoT+RAG prompt, generation got cut off mid-JSON. Fixed by explicitly setting `options: {"num_ctx": 8192, "num_predict": 2048}` in the Ollama request. Verified directly: the two prompts that previously exhausted all 3 retries via truncation succeeded 6/6 times when resent standalone after the fix.

Re-running the full N=25 sample after this fix: truncation-caused retries dropped from 9/13 to 2/9 — the fix works for its intended purpose. But the overall exhausted-failure rate went **up**, from 8.0% (2/25) to **12.0% (3/25)**:

Outcome	Round 1 (before fix)	Round 2 (after)
Clean on first attempt	12/25	19/25
Clean after a retry	11/25	3/25
Exhausted retry budget	2/25	3/25

Why more failures, not fewer, after fixing truncation. Truncation was previously cutting some responses off BEFORE the model finished writing a qualitative claim — meaning some of what looked like “just a truncation problem” was actually also masking a directional error that never got the chance to fully materialize (and get caught) in the truncated text. With generation now completing reliably, `_check_directional_consistency` sees the full text and catches more real errors than before — this is the check working correctly, not a regression.

All 3 round-2 exhausted failures share the exact same flag: `high_lv_ejection_fraction_suspected` (patient102, LV EF 70.9%; patient104, LV EF 84.4% — both across multiple repeats). Not random: MedGemma appears to have a systematic, reproducible tendency to under-characterize an elevated ejection fraction specifically — “reduced ejection fraction” is a far more common phrase in the model's training distribution than “elevated ejection fraction” (the rarer, HCM-associated finding this flag exists to catch), so the model defaults toward the familiar phrasing under uncertainty. This finding directly motivated Round 4's flag-specific anchor.

ROUND 1 — ORIGINAL FIX

The Original N=25 Result and the Two Deterministic Checks

The original 5-case test (Appendix, next page) found 2 real errors and 3 clean cases — but a single run cannot give a reliable error rate, since MedGemma's generation is non-deterministic. Confirmed directly while validating the fix: patient104, one of the 3 clean cases in the original run, produced a genuinely new characterization error (“...RV ejection fraction is slightly low at 65.6%” — not low, zero supporting citation) on a later re-run of the exact same input. **Errors are not tied to specific “hard” patients** — they can occur on any generation of any case. Methodology corrected accordingly: N=25 repeated generations (5 patients × 5 repeats, fixed MRI contribution per patient), not a single pass/fail.

The Fix, Implemented in sep_agent.py

- `_check_directional_consistency` — rejects a claim stating the opposite direction of what an already-raised flag establishes.
- `_check_ef_citation_coverage` — rejects an ejection-fraction claim with a concrete percentage that has no traceability entry backing it at all (added for the patient104 zero-citation gap the first check couldn't cover).

Both are deterministic, clause-level keyword heuristics (not NLI) that re-enter the same retry loop as a fabricated citation. Two real false positives were found and fixed while building these; one deliberate gap remains: a

self-contradiction expressed via negating the correct word in a different clause is not caught, favoring precision over recall for a check that gates a retry loop.

Appendix — Original 5-Case Test

Superseded by the N=25 results above; kept for the detailed case studies, which remain accurate as illustrations of the two originally-discovered failure modes.

Why the Existing Hallucination Check Couldn't Catch This

Traceability verification (`parse_findings_response`) confirms that a cited feature reference (`evidence_ref`) genuinely exists in the computed feature set — it says nothing about whether the qualitative word attached to that feature ("reduced", "elevated", "mild") correctly characterizes its actual value. Both failure cases below cite a real, correctly-valued feature; the citation-existence check therefore passed in both, by design.

Case 1 — Directional Misreading During Reasoning

patient102 · true diagnosis: NOR · LV ejection fraction 70.9%

- Ground truth: LV ejection fraction = 70.9% (*elevated* — this is exactly what triggered `high_lv_ejection_fraction_suspected`).
- CoT reasoning text: *"a decreased ejection fraction (70.9%)"*.
- CoT summary: *"ejection fraction reduced, raising suspicion for dilated cardiomyopathy"* — both the direction and the resulting suspected pathology are wrong.
- The no-CoT summary avoided this entirely, staying purely numeric — suggesting CoT's added reasoning step **introduced** this error rather than preventing it.

Case 2 — Contradiction Introduced During Compression

patient109 · true diagnosis: RV · RV ejection fraction 33.7%

- Ground truth: RV ejection fraction = 33.7% (~12 points below the review threshold).
- CoT reasoning text: **accurate** — *"the right ventricle has a low ejection fraction (33.7%)"*.
- CoT summary drifts within the same paragraph — *"mildly reduced..."* followed, one sentence later, by *"no signs of reduced ejection fraction"* — internally self-contradictory.
- Unlike Case 1, the reasoning step itself was **correct** here; the error was introduced during the reasoning-to-final-summary compression step.

Cases held up on inspection: the remaining 3 of 5 cases (DCM, MINF, HCM) showed correct direction, no internal contradiction, and valid traceability throughout.

Two Distinct Failure Modes

	Case 1 · patient102	Case 2 · patient109
Where the error appears	During reasoning itself	During summary compression, after correct reasoning
Effect of CoT	Introduced the error (no-CoT version was correct)	Reasoning was right; only the final summary drifted
Caught by traceability check?	No (citation is real)	No (citation is real)

Practical Takeaway

A valid traceability entry (`evidence_ref` resolves to a real feature) confirms the underlying data point is **not fabricated**. It does **NOT** confirm that the qualitative characterization built on top of it is directionally or internally consistent. Any narrative summary produced by this pipeline should be read with this distinction in mind, and is not a substitute for a clinician's own reading of the underlying numeric values.